

Optimizing GPU Cluster Utilization for ML Workloads

By: Diti Chhaproo | **Tools:** Python, PuLP (ILP), Discrete-Event Simulation

Problem

A shared 32-GPU cluster supporting 50-100 weekly ML training jobs experienced operational inefficiencies under first-come-first-served (FCFS) scheduling:

- **62% average GPU utilization** (significant idle capacity)
- **15% deadline miss rate** (high-priority jobs blocked by long batch jobs)
- **Unfair resource allocation** (experimental jobs monopolizing GPUs during peak hours)

Jobs varied by priority (production vs experimental), resource needs (1-8 GPUs), duration (30 min - 12 hours), and deadlines.

Approach

Baseline Analysis: FCFS scheduling showed GPU utilization gaps averaging 6 hours daily, with critical jobs waiting behind low-priority batch processes.

Bottleneck Identification: Pareto analysis revealed 3 job types consumed 70% of GPU-hours, with dependency chains creating unavoidable serialization.

Optimization Strategy:

1. **Formulated as Integer Linear Program** — Modeled job scheduling with constraints on GPU capacity, dependencies, and deadlines. Solved optimally for small instances to establish performance ceiling.
2. **Developed Priority-Weighted Greedy Heuristic** — Fast scheduling algorithm balancing job priority, resource efficiency (GPU-hours), and deadlines. Achieves 94% of optimal solution in <100ms vs 2+ minutes for ILP solver.

Key Tradeoff: Optimal scheduling (ILP) impractical for real-time decisions at scale. Greedy heuristic deployed for production with manual override capability for critical jobs.

Results

Metric	FCFS (Baseline)	Greedy Scheduler	Improvement
GPU Utilization	62%	81%	+31%
Deadline Miss Rate	15%	3%	-80%
Avg Job Latency	4.2 hrs	2.8 hrs	-34%

Validated via discrete-event simulation across 1,000 job scenarios. Greedy scheduler consistently achieved 90%+ of optimal performance while maintaining sub-second scheduling decisions.

Implementation & Tradeoffs

Decision: Deployed greedy heuristic rather than optimal ILP due to:

- Real-time scheduling requirements (2min solve time unacceptable)
- Scalability to 100+ concurrent jobs
- Acceptable near-optimal performance (94% vs optimal)

Acknowledged Risks:

- Low-priority jobs can experience starvation under high load
- Scheduler doesn't predict job duration (uses historical estimates)
- Fairness vs efficiency tension requires periodic manual rebalancing

Next Steps: Integrate ML-based duration prediction, explore multi-objective optimization for fairness constraints, evaluate job preemption for long-running processes.

Why This Matters

This demonstrates systems engineering judgment: knowing when optimal solutions are impractical and deploying near-optimal heuristics that work at scale. The approach generalizes to warehouse scheduling, cloud resource allocation, and manufacturing line balancing — any constrained resource optimization problem.

Technical Artifacts

GitHub repo includes: ILP formulation (PuLP), greedy scheduler implementation, discrete-event simulator, and comparative performance analysis.